

PLAN DE SOLUCIÓN INTEGRAL - SISTEMA DE VENTAS E INVENTARIO

CRTLPyme - Análisis Profesional y Propuesta de Implementación

Fecha: 21 de Noviembre de 2025

Versión: 1.0

Estado: Pendiente de Aprobación

ÍNDICE

1. [Resumen Ejecutivo](#)
 2. [Análisis del Estado Actual](#)
 3. [Arquitectura de Datos](#)
 4. [Problemas Identificados](#)
 5. [Consumidores de Datos](#)
 6. [Solución Propuesta](#)
 7. [Plan de Implementación](#)
 8. [Análisis de Riesgos](#)
 9. [Conclusiones y Recomendaciones](#)
-

RESUMEN EJECUTIVO

Situación Actual

El sistema CRTLPyme presenta una **inconsistencia estructural crítica** entre el schema de Prisma y la base de datos en producción. La tabla `inventory_movements` está definida en el schema pero **nunca fue migrada a producción**, causando que:

1.  Las ventas NO registran movimientos de inventario
2.  No hay trazabilidad de cambios de stock
3.  El sistema solo actualiza el campo `stock` sin registrar el historial
4.  La funcionalidad de movimientos está completamente deshabilitada

Impacto en el Negocio

- **Auditoría imposible:** No se puede rastrear qué causó un cambio en el inventario
- **Reportes limitados:** Sin datos de movimientos, los reportes de inventario son incompletos
- **Problemas de reconciliación:** Imposible detectar discrepancias entre ventas y stock
- **Riesgo operacional:** Sin trazabilidad para detectar errores o fraudes

Propuesta de Solución

Implementación de un **sistema completo de movimientos de inventario** que:

1. ☒ Crea la tabla `inventory_movements` en producción
2. ☒ Registra automáticamente movimientos en cada venta
3. ☒ Habilita la gestión manual de movimientos (entradas, salidas, ajustes)
4. ☒ Mantiene compatibilidad con todas las funcionalidades existentes
5. ☒ Garantiza integridad transaccional

ANÁLISIS DEL ESTADO ACTUAL

1. Estado de la Base de Datos

Tablas que SÍ existen en producción:

☒ <code>tenants</code>	- Multi-tenancy
☒ <code>users</code>	- Usuarios del sistema
☒ <code>master_products</code>	- Catálogo maestro de productos
☒ <code>tenant_inventory</code>	- Inventario específico por tenant
☒ <code>products_legacy</code>	- Tabla legacy (deprecated)
☒ <code>sales</code>	- Ventas realizadas
☒ <code>sale_items</code>	- Items de cada venta
☒ <code>stock_adjustments</code>	- Ajustes de inventario (limitado)
☒ <code>cash_sessions</code>	- Sesiones de caja
☒ <code>audit_logs</code>	- Logs de auditoría
☒ [... otras tablas ...]	

Tabla que NO existe en producción:

☒ <code>inventory_movements</code>	- Movimientos de inventario
--	-----------------------------

Evidencia:

- Archivo: `/prisma/migrations/20251106012548_complete_saas_implementation/migration.sql`
- La migración inicial NO incluye `CREATE TABLE "inventory_movements"`
- El schema actual (`prisma/schema.prisma`) SÍ define el modelo `InventoryMovement`
- **Conclusión:** Desincronización entre schema y base de datos

2. Análisis de Código

API de Ventas (`/app/api/sales/route.ts`)

Flujo actual al crear una venta:

```
//  LO QUE SÍ HACE:
1. Valida autenticación
2. Verifica sesión de caja abierta
3. Valida stock disponible
4. Calcula totales (subtotal, IVA, total)
5. Crea registro en tabla `sales`
6. Crea registros en tabla `sale_items`
7. Actualiza stock: decrement quantity en `tenant_inventory`
8. Registra en `audit_logs`

//  LO QUE NO HACE (CÓDIGO COMENTADO - líneas 271-286):
await tx.inventoryMovement.create({
  data: {
    tenantId: user.tenantId,
    tenantInventoryId: item.tenantInventoryId,
    type: 'EXIT',
    quantity: -item.quantity,
    reason: `Venta ${saleNumber}`,
    createdBy: user.id
  }
})
// Comentario: "La tabla 'inventory_movements' no existe en producción"
```

Problema identificado:

- El código para registrar movimientos existe pero está **comentado**
- La venta funciona, pero **sin trazabilidad** de por qué cambió el stock

API de Movimientos (/app/api/inventory/movements/route.ts)

Estado actual:

```
// GET - COMPLETAMENTE DESHABILITADO (líneas 30-46)
export async function GET(request: NextRequest) {
  return NextResponse.json(
    {
      movements: [],
      stats: { totalMovements: 0, /* ... */ },
      message: 'Los movimientos de inventario están temporalmente deshabilitados.'
    },
    { status: 200 }
  )
}

// POST - COMPLETAMENTE DESHABILITADO (líneas 159-167)
export async function POST(request: NextRequest) {
  return NextResponse.json(
    {
      error: 'Funcionalidad temporalmente deshabilitada',
      message: 'Los movimientos de inventario están en mantenimiento.'
    },
    { status: 503 }
  )
}

// TODO el código funcional está comentado (líneas 48-331)
```

Problema identificado:

- La funcionalidad completa está escrita y lista
- Pero está **completamente deshabilitada** por falta de la tabla

3. Schema de Prisma

Modelo InventoryMovement (líneas 185-205)

```

model InventoryMovement {
  id                String      @id @default(cuid())
  tenantInventoryId String      // ✓ Campo para FK
  type              MovementType // ENTRY, EXIT, ADJUSTMENT
  quantity          Int         // Cantidad movida
  reason            String?     // Motivo del movimiento
  notes             String?     // Notas adicionales
  createdBy         String      // userId
  tenantId          String
  createdAt         DateTime    @default(now())

  // Relations
  tenantInventory TenantInventory @relation(fields: [tenantInventoryId], references: [id])
  user            User           @relation(fields: [createdBy], references: [id])
  tenant          Tenant         @relation(fields: [tenantId], references: [id], onDelete: Cascade)

  @@index([tenantId, tenantInventoryId])
  @@index([tenantId, type])
  @@index([tenantId, createdAt])
  @@map("inventory_movements")
}

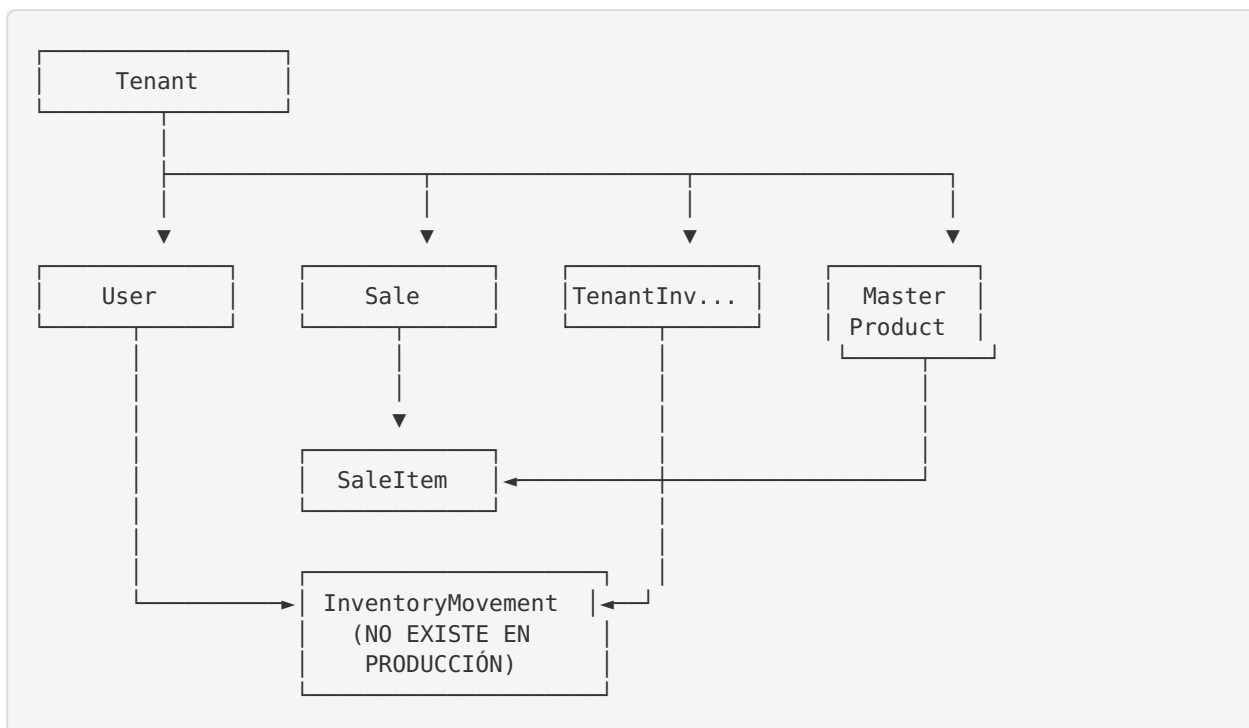
enum MovementType {
  ENTRY      // Entrada de stock (compras, reposiciones)
  EXIT       // Salida de stock (ventas)
  ADJUSTMENT // Ajuste de inventario (correcciones)
}

```

Análisis:

- ✓ Modelo bien diseñado con todas las relaciones necesarias
- ✓ Índices apropiados para queries eficientes
- ✓ Enum MovementType define claramente los tipos de movimientos
- ✓ Campos audit: createdBy, tenantId, createdAt
- ✗ **Tabla no existe en producción**

4. Relaciones de Modelos



Problema de relaciones:

- `TenantInventory` tiene una relación definida con `InventoryMovement`
- Pero al no existir la tabla, no se pueden crear movimientos
- Esto causa el error mostrado en la imagen del usuario:

`Invalid 'prisma.inventoryMovement.create()' invocation:`

`Argument 'tenantInventory' is missing`



ARQUITECTURA DE DATOS

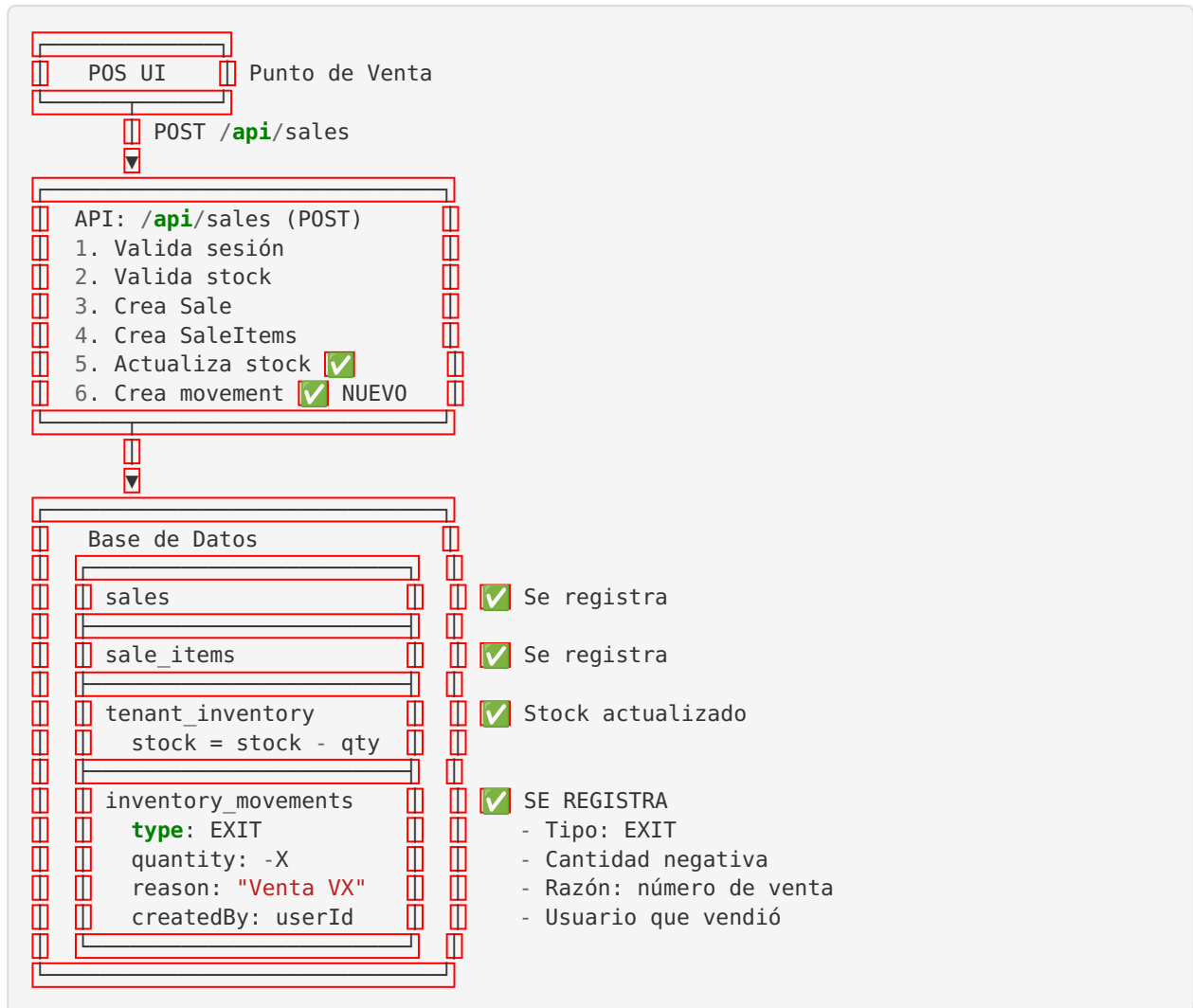
Flujo de Datos Actual (SIN movimientos)



Consecuencias:

- ☒ La venta se procesa correctamente
- ☒ El stock se reduce
- ☐ No hay registro de POR QUÉ se redujo el stock
- ☐ No hay trazabilidad
- ☐ Imposible auditar cambios de inventario

Flujo de Datos Propuesto (CON movimientos)



Beneficios:

- ✓ Trazabilidad completa de cambios de stock
- ✓ Auditoría de quién hizo qué y cuándo
- ✓ Base para reportes avanzados
- ✓ Detección de discrepancias
- ✓ Cumplimiento de normativas de auditoría



PROBLEMAS IDENTIFICADOS

Problema #1: Tabla inventory_movements no existe

Severidad: ● Crítica

Impacto: Alto

Descripción:

- El modelo `InventoryMovement` está definido en `prisma/schema.prisma`
- La migración inicial NO creó esta tabla en la base de datos
- Desincronización entre schema y base de datos real

Evidencia:

```
# Revisión de migraciones
$ grep -i "inventory_movement" prisma/migrations/*/migration.sql
# (Sin resultados - la tabla no se migró)

# Tablas existentes
$ grep "CREATE TABLE" prisma/migrations/*/migration.sql
# inventory_movements NO aparece en la lista
```

Consecuencias:

- Imposible crear registros de movimientos
- Código funcional deshabilitado
- Sin trazabilidad de cambios de inventario

Problema #2: Código de movimientos completamente deshabilitado

Severidad: ● Alta

Impacto: Alto

Descripción:

Dos áreas críticas de código están deshabilitadas:

A) En API de ventas (/app/api/sales/route.ts líneas 271-286):

```
// ⚠ TEMPORAL: inventoryMovement DESHABILITADO
// La tabla 'inventory_movements' no existe en producción.
// Se deshabilitó temporalmente para que las ventas funcionen.
// TODO: Implementar migración o eliminar feature completa.
/*
await tx.inventoryMovement.create({
  data: {
    tenantId: user.tenantId,
    tenantInventoryId: item.tenantInventoryId,
    type: 'EXIT',
    quantity: -item.quantity,
    reason: `Venta ${saleNumber}`,
    createdBy: user.id
  }
})
*/
```

B) En API de movimientos (/app/api/inventory/movements/route.ts):

- GET completo deshabilitado (retorna array vacío)
- POST completo deshabilitado (retorna error 503)
- ~280 líneas de código funcional comentadas

Consecuencias:

- Las ventas funcionan pero sin registro de movimientos
- Imposible gestionar entradas de inventario manualmente
- Imposible hacer ajustes de inventario con trazabilidad
- Sin historial de cambios

Problema #3: Sin trazabilidad de cambios de stock

Severidad: ● Alta

Impacto: Operacional y de Auditoría

Descripción:

Cuando se vende un producto:

```
// Solo se hace esto:
await tx.tenantInventory.update({
  where: { id: item.tenantInventoryId },
  data: {
    stock: { decrement: item.quantity }
  }
})

// ✗ NO se registra:
// - ¿Quién redujo el stock?
// - ¿Por qué se redujo? (venta, merma, robo, error)
// - ¿Cuándo exactamente?
// - ¿En qué venta específica?
```

Consecuencias:

- Imposible responder: "¿Por qué tengo 10 unidades menos de producto X?"
- Sin auditoría para detectar errores o fraudes
- Problemas de reconciliación de inventario
- Incumplimiento de normativas contables/fiscales

Problema #4: Gestión manual de inventario limitada

Severidad: ● Media

Impacto: Operacional

Descripción:

La tabla `stock_adjustments` existe pero es limitada:

```
model StockAdjustment {
  id String @id @default(cuid())
  tenantInventoryId String
  quantity Int // puede ser negativo
  type AdjustmentType // PURCHASE, LOSS, CORRECTION, RETURN
  reason String?
  userId String
  tenantId String
  createdAt DateTime @default(now())
  // ...
}
```

Problemas:

- ✗ No hay un flujo unificado de movimientos
- ✗ `StockAdjustment` es separado de `InventoryMovement`
- ✗ Duplicación de lógica
- ✗ Reportes fragmentados

Escenario problemático:

- Usuario hace ajuste manual → Se registra en `stock_adjustments`
- Sistema procesa venta → NO se registra movimiento
- Reporte de movimientos → Datos incompletos y fragmentados

Problema #5: Reportes e indicadores incompletos**Severidad:** ● Media**Impacto:** Análisis de Negocio**Descripción:**

Sin datos de movimientos, los reportes son limitados:

Reportes afectados:1. **Dashboard** (`/admin/dashboard/page.tsx`)

- Solo muestra stock actual
- No muestra tendencias de movimientos
- Sin alertas de movimientos anómalos

1. **Reportes de inventario**

- Sin historial de cambios
- Sin análisis de rotación de productos
- Sin detección de productos con movimientos sospechosos

2. **Análisis de ventas vs inventario**

- Imposible reconciliar ventas con cambios de stock
- Sin detección de discrepancias

Funcionalidad deseada pero no disponible:

- ✗ "Mostrar últimos 10 movimientos de este producto"
- ✗ "Productos con más salidas este mes"
- ✗ "Movimientos realizados por este usuario"
- ✗ "Detectar cambios de stock sin venta asociada"

**CONSUMIDORES DE DATOS****1. Punto de Venta (POS)****Archivo:** `/app/admin/pos/page.tsx`**Uso actual:**

```
// Al procesar una venta:
const response = await fetch('/api/sales', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    items: [...],
    paymentMethod: 'CASH',
    cashReceived: 10000
  })
})
```

Impacto de la solución:

- ☒ **NO requiere cambios** en el frontend
- ☒ La API sigue retornando el mismo formato
- ☒ El backend solo agregará registro de movimientos
- ☒ **Compatibilidad 100% garantizada**

2. Dashboard Administrativo

Archivo: /app/admin/dashboard/page.tsx

Uso actual:

```
const [inventoryRes, salesRes] = await Promise.all([
  fetch('/api/inventory'),
  fetch('/api/sales/stats?period=month')
])
```

Impacto de la solución:

- ☒ **NO requiere cambios** inmediatos
- ☐ **Mejora futura:** Poder agregar widget de “Últimos movimientos”
- ☐ **Mejora futura:** Alertas de movimientos anómalos

3. Reporte de Ventas

Archivo: /app/admin/reports/sales/page.tsx

Uso actual:

```
fetch('/api/sales?startDate=...&endDate=...')
```

Impacto de la solución:

- ☒ **NO requiere cambios**
- ☐ **Mejora futura:** Correlacionar ventas con movimientos de inventario

4. Gestión de Inventario

Archivo: /app/admin/inventory/page.tsx

Uso actual:

```
fetch('/api/inventory')
```

Impacto de la solución:

- ☒ **NO requiere cambios** en el listado actual
- ☐ **Nueva funcionalidad:** Ver historial de movimientos por producto

5. Movimientos de Inventario (Actualmente NO funcional)

Archivo: /app/admin/inventory/movements/page.tsx

Estado actual:

- Página existe pero muestra datos vacíos
- API retorna `movements: []`

Impacto de la solución:

- ☒ **SE HABILITARÁ** esta funcionalidad
- ☒ Permitirá ver historial de movimientos
- ☒ Permitirá registrar entradas y ajustes manuales

6. API de Estadísticas de Ventas

Archivo: /app/api/sales/stats/route.ts

Uso actual:

```
// Consulta ventas y calcula estadísticas
const sales = await prisma.sale.findMany({
  where: { tenantId, status: 'COMPLETED', createdAt: { gte: startDate } },
  include: { items: true }
})
```

Impacto de la solución:

- ☒ **NO requiere cambios**
- ☐ **Mejora futura:** Agregar estadísticas de movimientos

Resumen de Impacto en Consumidores

Consumidor	Cambios Re-queridos	Funcionalidad Afectada	Riesgo
POS	✗ Ninguno	✓ Solo mejora backend	● Cero
Dashboard	✗ Ninguno	● Nuevas opciones futuras	● Cero
Reportes de Ventas	✗ Ninguno	● Nuevas opciones futuras	● Cero
Inventario	✗ Ninguno	● Nuevas opciones futuras	● Cero
Movimientos	✓ Se habilita	✓ Funcionalidad nueva	● Cero
Stats API	✗ Ninguno	● Nuevas opciones futuras	● Cero

Conclusión: La solución es **100% retrocompatible** y no rompe ninguna funcionalidad existente.

✓ SOLUCIÓN PROPUESTA

Objetivo General

Implementar un sistema completo y robusto de movimientos de inventario que proporcione trazabilidad total, sin romper funcionalidades existentes.

Componentes de la Solución

1. Migración de Base de Datos

Objetivo: Crear la tabla `inventory_movements` en producción

Acción:

```
# Generar migración de Prisma
npx prisma migrate dev --name add_inventory_movements_table

# Aplicar en producción
npx prisma migrate deploy
```

Tabla a crear:

```

CREATE TABLE "inventory_movements" (
    "id" TEXT NOT NULL,
    "tenantInventoryId" TEXT NOT NULL,
    "type" TEXT NOT NULL,
    "quantity" INTEGER NOT NULL,
    "reason" TEXT,
    "notes" TEXT,
    "createdBy" TEXT NOT NULL,
    "tenantId" TEXT NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT "inventory_movements_pkey" PRIMARY KEY ("id")
);

CREATE INDEX "inventory_movements_tenantId_tenantInventoryId_idx"
    ON "inventory_movements" ("tenantId", "tenantInventoryId");
CREATE INDEX "inventory_movements_tenantId_type_idx"
    ON "inventory_movements" ("tenantId", "type");
CREATE INDEX "inventory_movements_tenantId_createdAt_idx"
    ON "inventory_movements" ("tenantId", "createdAt");

ALTER TABLE "inventory_movements"
    ADD CONSTRAINT "inventory_movements_tenantInventoryId_fkey"
    FOREIGN KEY ("tenantInventoryId")
    REFERENCES "tenant_inventory"("id")
    ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE "inventory_movements"
    ADD CONSTRAINT "inventory_movements_createdBy_fkey"
    FOREIGN KEY ("createdBy")
    REFERENCES "users"("id")
    ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE "inventory_movements"
    ADD CONSTRAINT "inventory_movements_tenantId_fkey"
    FOREIGN KEY ("tenantId")
    REFERENCES "tenants"("id")
    ON DELETE CASCADE ON UPDATE CASCADE;

```

Validación post-migración:

```

-- Verificar que la tabla existe
SELECT table_name FROM information_schema.tables
WHERE table_name = 'inventory_movements';

-- Verificar índices
SELECT indexname FROM pg_indexes
WHERE tablename = 'inventory_movements';

-- Verificar foreign keys
SELECT constraint_name FROM information_schema.table_constraints
WHERE table_name = 'inventory_movements' AND constraint_type = 'FOREIGN KEY';

```

2. Habilitación de Registro de Movimientos en Ventas

Objetivo: Cada venta debe registrar automáticamente sus movimientos de inventario

Archivo: /app/api/sales/route.ts

Cambio específico (líneas 271-286):

Código actual (comentado):

```
// ⚠ TEMPORAL: inventoryMovement DESHABILITADO
/*
await tx.inventoryMovement.create({
  data: {
    tenantId: user.tenantId,
    tenantInventoryId: item.tenantInventoryId,
    type: 'EXIT',
    quantity: -item.quantity,
    reason: `Venta ${saleNumber}`,
    createdBy: user.id
  }
})
*/
```

Código nuevo (habilitado):

```
// ✅ Registrar movimiento de inventario
await tx.inventoryMovement.create({
  data: {
    tenantId: user.tenantId,
    tenantInventoryId: item.tenantInventoryId,
    type: 'EXIT',
    quantity: -item.quantity,
    reason: `Venta ${saleNumber}`,
    createdBy: user.id
  }
})

console.log('✅ [SALES API] Movimiento de inventario registrado:', {
  type: 'EXIT',
  quantity: -item.quantity,
  reason: `Venta ${saleNumber}`
})
```

Validación:

- El registro de movimiento debe estar **dentro de la transacción**
- Si falla, toda la venta hace rollback
- Garantiza consistencia entre `sales`, `sale_items`, `tenant_inventory` y `inventory_movements`

3. Habilitación de API de Movimientos

Objetivo: Habilitar gestión manual de movimientos (entradas, ajustes, etc.)

Archivo: /app/api/inventory/movements/route.ts

Cambios:

A) GET - Listar movimientos:

```

// ANTES: Retornaba array vacío
return NextResponse.json({
  movements: [],
  message: 'Temporalmente deshabilitados'
})

// DESPUÉS: Habilitar código funcional (descomentar líneas 48-147)
const movements = await prisma.inventoryMovement.findMany({
  where,
  include: {
    tenantInventory: {
      select: {
        id: true,
        customSku: true,
        masterProduct: { select: { id: true, sku: true, name: true } }
      }
    },
    user: {
      select: { id: true, firstName: true, lastName: true }
    }
  },
  orderBy: { createdAt: 'desc' },
  take: limit
})

return NextResponse.json({
  movements,
  stats: { /* estadísticas calculadas */ }
})

```

B) POST - Crear movimiento manual:


```
// ANTES: Retornaba error 503
return NextResponse.json(
  { error: 'Funcionalidad deshabilitada' },
  { status: 503 }
)

// DESPUÉS: Habilitar código funcional (descomentar líneas 169-330)
const movement = await prisma.$transaction(async (tx) => {
  // 1. Crear movimiento
  const newMovement = await tx.inventoryMovement.create({
    data: {
      tenantInventoryId: validatedData.tenantInventoryId,
      type: validatedData.type,
      quantity: quantityChange,
      reason: validatedData.reason,
      notes: validatedData.notes,
      createdBy: session.user.id,
      tenantId: session.user.tenantId
    }
  })

  // 2. Actualizar stock
  await tx.tenantInventory.update({
    where: { id: validatedData.tenantInventoryId },
    data: { stock: newStock }
  })

  return newMovement
})

return NextResponse.json(movement, { status: 201 })
```

4. Validaciones y Seguridad

A) Validación de datos:

```
const createMovementSchema = z.object({
  tenantInventoryId: z.string().min(1),
  type: z.enum(['ENTRY', 'EXIT', 'ADJUSTMENT']),
  quantity: z.number().int().positive(),
  reason: z.string().min(1).max(255),
  notes: z.string().max(500).optional()
})
```

B) Validación de permisos:

```
// Solo ADMIN, INVENTARIO y PROVEEDOR pueden crear movimientos manuales
if (!['ADMIN', 'INVENTARIO', 'PROVEEDOR'].includes(session.user.role)) {
  return NextResponse.json(
    { error: 'No tienes permisos para registrar movimientos' },
    { status: 403 }
  )
}
```

C) Validación de stock:

```
// No permitir stock negativo
const newStock = inventoryItem.stock + quantityChange
if (newStock < 0) {
  return NextResponse.json(
    {
      error: `Stock insuficiente. Stock actual: ${inventoryItem.stock}`,
      currentStock: inventoryItem.stock
    },
    { status: 400 }
  )
}
```

D) Validación de tenant:

```
// Asegurar que el producto pertenece al tenant del usuario
const inventoryItem = await prisma.tenantInventory.findFirst({
  where: {
    id: validatedData.tenantInventoryId,
    tenantId: session.user.tenantId, // ✅ Multi-tenancy
    isActive: true
  }
})
```

5. Auditoría

Registro en audit_logs:

```
// Cada movimiento registra en auditoría
await prisma.auditLog.create({
  data: {
    action: 'CREATE',
    entity: 'InventoryMovement',
    entityId: movement.id,
    newValues: {
      ...movement,
      previousStock: inventoryItem.stock,
      newStock
    },
    userId: session.user.id,
    tenantId: session.user.tenantId
  }
})
```

Casos de Uso Cubiertos

Caso 1: Venta en POS

Usuario vende 3 unidades de producto X



Sistema:

1. Crea venta
2. Crea sale_items
3. Reduce stock: 100 → 97
4. ☒ NUEVO: Registra movimiento:
 - **type**: EXIT
 - quantity: -3
 - reason: "Venta V-000123"
 - createdBy: userId

Caso 2: Entrada de inventario (compra)

Usuario registra compra de 50 unidades



POST /**api**/inventory/movements

```
{
  "tenantInventoryId": "abc123",
  "type": "ENTRY",
  "quantity": 50,
  "reason": "Compra proveedor XYZ",
  "notes": "Factura #12345"
}
```



Sistema:

1. Valida permisos
2. Incrementa stock: 97 → 147
3. Registra movimiento
4. Registra en auditoría

Caso 3: Ajuste por merma

Usuario registra pérdida de 2 unidades (rotas)



POST /**api**/inventory/movements

```
{
  "tenantInventoryId": "abc123",
  "type": "ADJUSTMENT",
  "quantity": -2,
  "reason": "Merma - productos dañados",
  "notes": "Encontrados rotos en almacén"
}
```



Sistema:

1. Valida permisos
2. Reduce stock: 147 → 145
3. Registra movimiento
4. Registra en auditoría

Caso 4: Consultar historial de un producto

GET `/api/inventory/movements?tenantInventoryId=abc123&limit=50`



Sistema retorna:

```
[
  {
    "id": "mov001",
    "type": "EXIT",
    "quantity": -3,
    "reason": "Venta V-000123",
    "createdAt": "2025-11-21T10:30:00Z",
    "user": { "firstName": "Juan", "lastName": "Pérez" }
  },
  {
    "id": "mov002",
    "type": "ENTRY",
    "quantity": 50,
    "reason": "Compra proveedor XYZ",
    "createdAt": "2025-11-20T14:00:00Z",
    "user": { "firstName": "María", "lastName": "González" }
  },
  // ...
]
```



PLAN DE IMPLEMENTACIÓN

Fase 1: Preparación (1-2 horas)

Responsable: Desarrollador

Objetivo: Preparar entorno y validar cambios

Tareas:

1. ☒ **Backup de base de datos de producción**

bash

```
# Cloud SQL Backup
```

```
gcloud sql backups create --instance=INSTANCE_NAME
```

2. ☒ **Crear branch de desarrollo**

bash

```
git checkout -b feature/inventory-movements-implementation
```

3. ☒ **Documentar estado actual**

- Screenshot de reportes actuales
- Screenshot de dashboard
- Exportar datos de `tenant_inventory` para validación posterior

4. ☒ **Revisar código a modificar**

- `/app/api/sales/route.ts`
- `/app/api/inventory/movements/route.ts`
- Confirmar números de línea

Fase 2: Migración de Base de Datos (30 minutos)

Responsable: Desarrollador

Objetivo: Crear tabla `inventory_movements`

Tareas:

1. **Generar migración de Prisma**

```
bash
cd /home/ubuntu/CRTLPyne
npx prisma migrate dev --name add_inventory_movements_table --create-only
```

2. **Revisar migración generada**

```
bash
cat prisma/migrations/[timestamp]_add_inventory_movements_table/migration.sql
```

- Verificar que crea tabla correcta
- Verificar índices
- Verificar foreign keys

3. **Probar migración en desarrollo local (si disponible)**

```
bash
npx prisma migrate dev
```

4. **Aplicar migración en producción**

```
```bash
Opción 1: Mediante CI/CD (recomendado)
git add prisma/migrations/
git commit -m "feat: add inventory_movements table"
git push origin feature/inventory-movements-implementation
→ GitHub Actions desplegará automáticamente
```

# Opción 2: Manual (si CI/CD falla)

```
npx prisma migrate deploy
```
```

1. **Validar migración**

```
```sql
- Conectar a Cloud SQL y verificar
SELECT table_name FROM information_schema.tables
WHERE table_name = 'inventory_movements';

SELECT column_name, data_type FROM information_schema.columns
WHERE table_name = 'inventory_movements';
```
```

Criterios de éxito:

-  Tabla `inventory_movements` existe
 -  Todos los índices creados
 -  Foreign keys funcionando
 -  Sin errores en logs de Cloud SQL
-

Fase 3: Habilitación de Código (1 hora)

Responsable: Desarrollador

Objetivo: Descomentar y habilitar código funcional

Subtarea 3.1: API de Ventas

Archivo: /app/api/sales/route.ts

Cambios:

1. Líneas 271-286: Descomentar bloque de `inventoryMovement.create()`
2. Agregar logging para debug:

```
typescript
console.log('✅ [SALES API] Movimiento de inventario registrado:', {
  type: 'EXIT',
  quantity: -item.quantity,
  productId: item.tenantInventoryId
})
```

3. Actualizar comentario de “TEMPORAL” a “HABILITADO”

Archivo modificado:

```
git add app/api/sales/route.ts
git commit -m "feat: enable inventory movements in sales API"
```

Subtarea 3.2: API de Movimientos - GET

Archivo: /app/api/inventory/movements/route.ts

Cambios:

1. Líneas 30-46: Eliminar código que retorna array vacío
2. Líneas 48-147: Descomentar código funcional completo
3. Agregar logging:

```
typescript
console.log('✅ [MOVEMENTS API] Movimientos recuperados:', movements.length)
```

Archivo modificado:

```
git add app/api/inventory/movements/route.ts
git commit -m "feat: enable inventory movements GET endpoint"
```

Subtarea 3.3: API de Movimientos - POST

Archivo: /app/api/inventory/movements/route.ts

Cambios:

1. Líneas 159-167: Eliminar código que retorna error 503
2. Líneas 169-330: Descomentar código funcional completo
3. Agregar logging detallado:

```
typescript
console.log('✅ [MOVEMENTS API] Movimiento creado:', {
  id: movement.id,
  type: validatedData.type,
  quantity: quantityChange,
```

```

    previousStock: inventoryItem.stock,
    newStock
  })

```

Archivo modificado:

```

git add app/api/inventory/movements/route.ts
git commit -m "feat: enable inventory movements POST endpoint"

```

Fase 4: Testing Exhaustivo (2-3 horas)

Responsable: Desarrollador + Usuario

Objetivo: Validar que todo funciona correctamente

Test 1: Venta Simple

Objetivo: Verificar que las ventas registran movimientos

Pasos:

1. Abrir sesión de caja
2. Verificar stock inicial de un producto:

```

sql
SELECT stock FROM tenant_inventory WHERE id = 'xxx';
-- Ej: stock = 100

```

3. Vender 3 unidades de ese producto en POS

4. Validar:

```

```sql
- Stock actualizado
SELECT stock FROM tenant_inventory WHERE id = 'xxx';
- Debe ser: 97

```

- Movimiento registrado

```

SELECT * FROM inventory_movements
WHERE "tenantInventoryId" = 'xxx'
ORDER BY "createdAt" DESC LIMIT 1;

```

- Debe existir registro:
- type: 'EXIT'
- quantity: -3
- reason: 'Venta V-xxxxxx'

```

```

```

Resultado esperado:

- ☒ Venta se procesa correctamente
- ☒ Stock se reduce de 100 a 97
- ☒ Movimiento registrado con type='EXIT', quantity=-3
- ☒ Sin errores en logs

Test 2: Entrada de Inventario

Objetivo: Verificar que se pueden registrar compras

Pasos:

1. Llamar a API:

bash

```
curl -X POST http://localhost:3000/api/inventory/movements \
  -H "Content-Type: application/json" \
  -H "Cookie: next-auth.session-token=..." \
  -d '{
    "tenantInventoryId": "xxx",
    "type": "ENTRY",
    "quantity": 50,
    "reason": "Compra proveedor TEST",
    "notes": "Test de entrada"
  }'
```

1. Validar respuesta:

json

```
{
  "id": "mov_xxx",
  "type": "ENTRY",
  "quantity": 50,
  "previousStock": 97,
  "newStock": 147,
  ...
}
```

2. Validar en base de datos:

``sql

```
SELECT stock FROM tenant_inventory WHERE id = 'xxx';
- Debe ser: 147
```

```
SELECT * FROM inventory_movements
```

```
WHERE "tenantInventoryId" = 'xxx'
```

```
ORDER BY "createdAt" DESC LIMIT 1;
```

```
- Debe existir registro con quantity=50
```

```
...
```

Resultado esperado:

- ☒ API responde 201 Created
- ☒ Stock se incrementa de 97 a 147
- ☒ Movimiento registrado correctamente
- ☒ Audit log registrado

Test 3: Ajuste Negativo**Objetivo:** Verificar mermas/correcciones negativas**Pasos:**

1. Llamar a API:

bash

```
curl -X POST http://localhost:3000/api/inventory/movements \
  -H "Content-Type: application/json" \
```



```
-H "Cookie: next-auth.session-token=..." \
-d '{
  "tenantInventoryId": "xxx",
  "type": "ADJUSTMENT",
  "quantity": -5,
  "reason": "Merma - productos dañados",
  "notes": "Test de ajuste negativo"
}'
```

1. Validar:

```
```sql
SELECT stock FROM tenant_inventory WHERE id = 'xxx';
- Debe ser: 142 (147 - 5)
```

```
SELECT * FROM inventory_movements WHERE type = 'ADJUSTMENT';
- Debe existir con quantity=-5
```
```

Resultado esperado:

- ☒ Stock se reduce correctamente
- ☒ Movimiento tipo ADJUSTMENT registrado
- ☒ No se permite stock negativo

Test 4: Validación de Stock Negativo

Objetivo: Verificar que no se permite stock negativo

Pasos:

1. Obtener stock actual:

```
sql
SELECT stock FROM tenant_inventory WHERE id = 'xxx';
-- Ej: stock = 142
```

1. Intentar ajuste que resulte en negativo:

```
bash
curl -X POST http://localhost:3000/api/inventory/movements \
-d '{
  "tenantInventoryId": "xxx",
  "type": "ADJUSTMENT",
  "quantity": -200,
  "reason": "Test stock negativo"
}'
```

2. Validar respuesta:

```
json
{
  "error": "Stock insuficiente. Stock actual: 142",
  "currentStock": 142
}
```

Resultado esperado:

- ☒ API rechaza con status 400

- ☒ Stock NO cambia
- ☒ NO se crea movimiento
- ☒ Mensaje de error claro

Test 5: Consulta de Historial

Objetivo: Verificar que se puede consultar historial de movimientos

Pasos:

1. Llamar a API GET:

```
bash
```

```
curl http://localhost:3000/api/inventory/movements?tenantInventoryId=xxx&limit=10
```

1. Validar respuesta:

```
json
{
  "movements": [
    {
      "id": "mov_001",
      "type": "EXIT",
      "quantity": -3,
      "reason": "Venta V-000123",
      "createdAt": "2025-11-21T10:30:00Z",
      "user": { "firstName": "Juan", "lastName": "Pérez" },
      "tenantInventory": { ... }
    },
    // ... más movimientos
  ],
  "stats": {
    "totalMovements": 15,
    "entriesCount": 5,
    "exitsCount": 8,
    "adjustmentsCount": 2,
    ...
  }
}
```

Resultado esperado:

- ☒ API retorna lista de movimientos
- ☒ Ordenados por fecha DESC (más reciente primero)
- ☒ Incluye datos de usuario y producto
- ☒ Estadísticas correctas

Test 6: Multi-Tenancy

Objetivo: Verificar que un tenant no puede ver movimientos de otro

Pasos:

1. Loguearse como Tenant A
2. Obtener movimientos:

```
bash
```

```
curl http://localhost:3000/api/inventory/movements
```

3. Loguearse como Tenant B

4. Obtener movimientos:

```
bash
```

```
curl http://localhost:3000/api/inventory/movements
```

5. Comparar: Los movimientos deben ser diferentes

Resultado esperado:

- ☒ Cada tenant solo ve sus propios movimientos
- ☒ Aislamiento completo de datos

Test 7: Validación de Permisos

Objetivo: Verificar que solo usuarios autorizados pueden crear movimientos

Pasos:

1. Loguearse como usuario CAJA (sin permisos)

2. Intentar crear movimiento:

```
bash
```

```
curl -X POST http://localhost:3000/api/inventory/movements \
-d '{ "tenantInventoryId": "xxx", "type": "ENTRY", ... }'
```

1. Validar respuesta:

```
json
```

```
{
```

```
  "error": "No tienes permisos para registrar movimientos de inventario"
```

```
}
```

Resultado esperado:

- ☒ API rechaza con status 403
- ☒ NO se crea movimiento
- ☒ Mensaje de error claro

Test 8: Integridad Transaccional

Objetivo: Verificar rollback en caso de error

Pasos:

1. Simular error en medio de transacción (modificar código temporalmente para lanzar error después de crear movimiento pero antes de actualizar stock)

2. Intentar crear movimiento

3. Validar:

```
```sql
```

```
SELECT COUNT(*) FROM inventory_movements WHERE reason = 'Test rollback';
```

- Debe ser: 0 (rollback exitoso)

```
SELECT stock FROM tenant_inventory WHERE id = 'xxx';
```

- Stock NO debe haber cambiado

```
```
```

Resultado esperado:

- ☒ Transacción hace rollback completo
 - ☒ NO se crea movimiento parcial
 - ☒ Stock no cambia
-

Fase 5: Validación con Usuario (1-2 horas)

Responsable: Usuario con soporte de Desarrollador

Objetivo: Validar funcionalidad end-to-end en escenario real

Tareas:

1. **Usuario realiza ventas normales**
 - Procesar al menos 5 ventas diferentes
 - Verificar que todas se completan sin errores
 - Verificar que recibos se imprimen correctamente
2. **Usuario consulta historial de movimientos**
 - Ir a `/admin/inventory/movements`
 - Verificar que aparecen las ventas realizadas
 - Filtrar por producto específico
3. **Usuario registra entrada de inventario**
 - Registrar compra de productos
 - Verificar que stock se actualiza
 - Verificar que movimiento aparece en historial
4. **Usuario valida reportes**
 - Dashboard debe seguir funcionando
 - Reportes de ventas deben seguir funcionando
 - Inventario debe mostrar stocks correctos

Criterios de aprobación:

- ☒ Todas las ventas funcionan sin errores
 - ☒ Movimientos se registran correctamente
 - ☒ Historial es claro y útil
 - ☒ No hay errores en consola del navegador
 - ☒ Dashboard y reportes funcionan normalmente
-

Fase 6: Despliegue a Producción (30 minutos)

Responsable: Desarrollador

Objetivo: Llevar cambios a producción de forma segura

Tareas:

1. ☒ **Crear Pull Request**

```
bash
git push origin feature/inventory-movements-implementation
# Crear PR en GitHub
```

2. **Code Review (opcional pero recomendado)**

- Revisar cambios línea por línea
- Verificar que solo se modificaron archivos necesarios

3. **Merge a main**

```
bash
git checkout main
git pull origin main
git merge feature/inventory-movements-implementation
git push origin main
```

4. **Verificar despliegue automático**

- GitHub Actions debe iniciar build
- Cloud Build debe desplegar a Cloud Run
- Monitoring: verificar que no hay errores

5. **Verificar en producción**

```
bash
curl https://crtlpyme-xxxxxxx.run.app/api/inventory/movements
```

6. **Smoke tests en producción**

- Procesar una venta de prueba
- Verificar que movimiento se registra
- Consultar historial de movimientos

Criterios de éxito:

-  Despliegue sin errores
-  Aplicación responde correctamente
-  Venta de prueba funciona y registra movimiento
-  Sin errores en logs de Cloud Run

Fase 7: Monitoreo Post-Despliegue (24-48 horas)

Responsable: Desarrollador + Usuario

Objetivo: Asegurar estabilidad y detectar problemas temprano

Tareas:

1. Monitoreo de logs (primeras 2 horas)

```
bash
gcloud logging read "resource.type=cloud_run_revision" \
  --project=PROJECT_ID \
  --limit=100 \
  --format=json
```

- Buscar errores relacionados con `inventory_movements`
- Verificar que ventas se procesan correctamente

2. Monitoreo de performance

- Verificar tiempos de respuesta de `/api/sales`
- Verificar tiempos de respuesta de `/api/inventory/movements`
- Alertar si hay degradación

3. Validación de datos (después de 24 horas)

```
```sql
```

- Cantidad de movimientos registrados

```
SELECT COUNT(*) FROM inventory_movements;
```

- Movimientos por tipo

```
SELECT type, COUNT(*) FROM inventory_movements GROUP BY type;
```

- Verificar que todas las ventas tienen movimientos

```
SELECT
```

```
(SELECT COUNT() FROM sales WHERE "createdAt" > NOW() - INTERVAL '24 hours') as ventas,
```

```
(SELECT COUNT() FROM inventory_movements WHERE type = 'EXIT' AND "createdAt" > NOW() -
INTERVAL '24 hours') as movimientos;
```

- Deben ser iguales o muy cercanos

```
```
```

1. Feedback del usuario

- ¿Las ventas funcionan sin problemas?

- ¿El historial de movimientos es útil?

- ¿Hay alguna funcionalidad que no funciona como antes?

Señales de alerta (requieren acción inmediata):

- 🚨 Errores 500 en `/api/sales`
- 🚨 Ventas que no se completan
- 🚨 Discrepancias entre ventas y movimientos
- 🚨 Performance degradada (>2x tiempo de respuesta)
- 🚨 Quejas de usuarios



ANÁLISIS DE RIESGOS

Riesgo 1: Migración de base de datos falla

Probabilidad: ● Baja

Impacto: ● Crítico

Severidad: ● Media

Descripción:

La migración de Prisma falla al crear la tabla `inventory_movements` en Cloud SQL.

Posibles causas:

- Problemas de conectividad con Cloud SQL
- Permisos insuficientes del usuario de base de datos
- Sintaxis SQL incompatible con versión de PostgreSQL
- Timeout en Cloud SQL

Mitigación PREVIA:

1. ✅ Hacer backup completo de base de datos antes de iniciar
2. ✅ Probar migración en entorno de desarrollo local primero
3. ✅ Verificar permisos del usuario de base de datos:

```
sql
```

```
SELECT * FROM information_schema.role_table_grants
```

```
WHERE grantee = 'db_user';
```

4.  Revisar migración generada manualmente antes de aplicar

Plan de recuperación:

1. Si la migración falla:

```
bash
```

```
npx prisma migrate resolve --rolled-back [migration_name]
```

2. Revisar logs de error detallados
3. Corregir problema identificado
4. Reintentar migración
5. Si no se puede resolver: restaurar backup

Tiempo estimado de recuperación: 30-60 minutos

Riesgo 2: Ventas fallan después de habilitar movimientos

Probabilidad:  Media

Impacto:  Crítico

Severidad:  Alta

Descripción:

Al descomentar código de `inventoryMovement.create()` en la API de ventas, las transacciones empiezan a fallar.

Posibles causas:

- Error en sintaxis del código descomentado
- Problema de relaciones de Prisma
- Campos requeridos faltantes
- Error de validación

Mitigación PREVIA:

1.  Revisar cuidadosamente código antes de descomentar
2.  Agregar try-catch específico alrededor del create:

```
typescript
```

```
try {
  await tx.inventoryMovement.create({ ... })
} catch (movementError) {
  console.error('Error al crear movimiento:', movementError)
  throw movementError // Propaga error para rollback
}
```

3.  Agregar logging exhaustivo
4.  Probar en ambiente de desarrollo primero

Plan de recuperación:

1. Si las ventas empiezan a fallar:

```
bash
```

```
# Rollback inmediato
```

```
git revert HEAD
```

```
git push origin main
```

2. GitHub Actions desplegará versión anterior automáticamente
3. Analizar logs de error

4. Corregir problema
5. Re-desplegar con fix

Tiempo estimado de recuperación: 10-20 minutos (rollback) + tiempo de fix

Riesgo 3: Performance degradada

Probabilidad: ● Baja

Impacto: ● Medio

Severidad: ● Baja

Descripción:

La creación adicional de registros en `inventory_movements` ralentiza las ventas.

Análisis de impacto:

```
// Antes: 2 escrituras por venta
await tx.sale.create({ ... }) // 1 write
await tx.saleItem.createMany({ ... }) // 1 write (batch)
await tx.tenantInventory.update({ ... }) // 1 write

// Después: +1 escritura por item vendido
await tx.inventoryMovement.create({ ... }) // +N writes (N = cantidad de items)
```

Escenario peor caso:

- Venta con 10 items diferentes
- Antes: ~3 escrituras
- Después: ~13 escrituras (3 + 10 movimientos)
- Incremento: ~4x en cantidad de escrituras

Mitigación PREVIA:

1. ☒ Usar transacciones (ya implementado)
2. ☒ Índices apropiados en `inventory_movements` (ya definidos en schema)
3. ☒ Batch inserts donde sea posible:

```
typescript
// Opción de optimización futura si es necesario
await tx.inventoryMovement.createMany({
  data: items.map(item => ({
    tenantId,
    tenantInventoryId: item.id,
    type: 'EXIT',
    quantity: -item.quantity,
    reason: `Venta ${saleNumber}`,
    createdBy: user.id
  }))
})
```

Monitoreo:

- Medir tiempos de respuesta de `/api/sales` antes y después
- Alertar si tiempo de respuesta > 2 segundos
- Meta: mantener tiempo de respuesta < 1 segundo

Plan de recuperación:

1. Si performance es inaceptable:

- Implementar batch inserts
- Optimizar índices
- Considerar hacer inserts async (fuera de transacción crítica)

Tiempo estimado de recuperación: 2-4 horas (optimización)

Riesgo 4: Datos inconsistentes (ventas sin movimientos)

Probabilidad: ● Media

Impacto: ● Medio

Severidad: ● Media

Descripción:

Si hay un error parcial, podrían quedar ventas registradas sin sus movimientos correspondientes.

Escenario:

```
await tx.sale.create({ ... }) // ✓ Éxito
await tx.saleItem.create({ ... }) // ✓ Éxito
await tx.inventoryMovement.create({ ... }) // ✗ Falla
// → Transacción hace rollback
// → PERO si hay un bug, podría no hacer rollback completo
```

Mitigación PREVIA:

1. ✓ TODO el código debe estar dentro de `prisma.$transaction()`

2. ✓ Agregar validación post-transacción:

```
```typescript
```

```
const sale = await prisma.$transaction(async (tx) => {
```

```
// ... todas las operaciones
```

```
return newSale
```

```
})
```

```
// Validar que se creó el movimiento
```

```
const movementCount = await prisma.inventoryMovement.count({
```

```
where: {
```

```
reason: Venta ${saleNumber}
```

```
}
```

```
})
```

```
if (movementCount === 0) {
```

```
console.error('⚠️ ADVERTENCIA: Venta creada sin movimientos')
```

```
}
```

```
```
```

Detección:

Script de validación diario:

```
-- Ventas sin movimientos correspondientes (últimas 24 horas)
SELECT s.id, s."saleNumber", s."createdAt"
FROM sales s
LEFT JOIN sale_items si ON si."saleId" = s.id
LEFT JOIN inventory_movements im ON im.reason = 'Venta ' || s."saleNumber"
WHERE s."createdAt" > NOW() - INTERVAL '24 hours'
      AND im.id IS NULL
GROUP BY s.id;
```

Plan de recuperación:

1. Identificar ventas afectadas

2. Script de corrección:

```
```typescript
// Para cada venta sin movimientos
const saleItems = await prisma.saleItem.findMany({
 where: { saleId: 'xxx' }
})

await prisma.inventoryMovement.createMany({
 data: saleItems.map(item => ({
 tenantId: sale.tenantId,
 tenantInventoryId: item.tenantInventoryId,
 type: 'EXIT',
 quantity: -item.quantity,
 reason: `Venta ${sale.saleNumber} (corrección)`,
 createdBy: sale.userId,
 createdAt: sale.createdAt // Mantener fecha original
 }))
})
```
```

Tiempo estimado de recuperación: 1-2 horas

Riesgo 5: Romper funcionalidades existentes

Probabilidad:  Muy Baja

Impacto:  Alto

Severidad:  Baja

Descripción:

Los cambios afectan negativamente a reportes, dashboard u otras funcionalidades.

Análisis:

-  NO se modifican estructuras de datos existentes
-  NO se cambian respuestas de APIs existentes
-  Solo se AGREGAN nuevos registros (movimientos)
-  Solo se HABILITA código ya existente

Áreas de riesgo mínimo:

- Dashboard: Solo consulta `sales` y `tenant_inventory` (sin cambios)
- Reportes de ventas: Solo consulta `sales` (sin cambios)

- POS: Solo llama a `/api/sales` POST (respuesta sin cambios)
- Inventario: Solo consulta `tenant_inventory` (sin cambios)

Mitigación PREVIA:

1. ☒ NO modificar estructura de respuestas de APIs existentes
2. ☒ Mantener backwards compatibility 100%
3. ☒ Agregar tests de regresión antes de desplegar

Validación post-despliegue:

```
# Checklist de funcionalidades a validar
☒ POS: Crear venta
☒ Dashboard: Ver estadísticas
☒ Reportes: Generar reporte de ventas
☒ Inventario: Listar productos
☒ Sesiones de caja: Abrir/cerrar sesión
☒ Usuarios: Login/logout
```

Plan de recuperación:

Si algo se rompe:

```
git revert HEAD
git push origin main
# → Despliegue automático de versión anterior
```

Tiempo estimado de recuperación: 5-10 minutos

Riesgo 6: Problemas de multi-tenancy

Probabilidad: ● Muy Baja

Impacto: ● Crítico (si ocurre)

Severidad: ● Baja

Descripción:

Un tenant podría ver o modificar movimientos de otro tenant.

Mitigación PREVIA:

1. ☒ Todos los queries filtran por `tenantId`:

```
typescript
where: {
  tenantId: session.user.tenantId, // ☒ Siempre presente
  ...
}
```

1. ☒ Validaciones en cada endpoint:

```
``typescript
const inventoryItem = await prisma.tenantInventory.findFirst({
  where: {
    id: tenantInventoryId,
    tenantId: session.user.tenantId // ☒ Valida ownership
```

```

    }
  })
  if (!inventoryItem) {
    return NextResponse.json({ error: 'No encontrado' }, { status: 404 })
  }
  ...

```

1.  Row-Level Security (RLS) en PostgreSQL (opcional pero recomendado):

```

```sql
ALTER TABLE inventory_movements ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON inventory_movements
USING ("tenantId" = current_setting('app.current_tenant')::text);
```

```

Validación:

Test de penetración:

```

# Loguearse como Tenant A
curl -X GET /api/inventory/movements \
-H "Cookie: session_tenant_a"

# Intentar acceder a datos de Tenant B
curl -X POST /api/inventory/movements \
-H "Cookie: session_tenant_a" \
-d '{ "tenantInventoryId": "ID_DE_TENANT_B", ... }'
# Debe retornar 404 o 403

```

Plan de recuperación:

Si se detecta violación de multi-tenancy:

1.  Despliegue inmediato de hotfix
2. Auditar registros creados/modificados
3. Corregir datos afectados
4. Notificar a tenants afectados

Tiempo estimado de recuperación: 1-2 horas (crítico)

Matriz de Riesgos (Resumen)

| Riesgo | Probabilidad | Impacto | Severidad | Tiempo Recuperación |
|-----------------------|--------------|-----------|-----------|---------------------|
| Migración falla | ● Baja | ● Crítico | ● Media | 30-60 min |
| Ventas fallan | ● Media | ● Crítico | ● Alta | 10-20 min |
| Performance degradada | ● Baja | ● Medio | ● Baja | 2-4 horas |
| Datos inconsistentes | ● Media | ● Medio | ● Media | 1-2 horas |
| Funcionalidades rotas | ● Muy Baja | ● Alto | ● Baja | 5-10 min |
| Multi-tenancy violado | ● Muy Baja | ● Crítico | ● Baja | 1-2 horas |

Evaluación general: ● RIESGO ACEPTABLE

La mayoría de riesgos tienen probabilidad baja y planes de recuperación claros.



BENEFICIOS ESPERADOS

Beneficios Operacionales

1. Trazabilidad Completa

- ✓ Saber exactamente qué causó cada cambio de stock
- ✓ Identificar quién realizó cada operación
- ✓ Fecha y hora exacta de cada movimiento

2. Auditoría y Cumplimiento

- ✓ Cumplimiento de normativas contables
- ✓ Historial inmutable de movimientos
- ✓ Soporte para auditorías internas/externas

3. Detección de Problemas

- ✓ Identificar discrepancias entre ventas y stock
- ✓ Detectar errores de registro
- ✓ Alertar sobre movimientos anómalos

4. Gestión de Inventario Mejorada

- ✓ Registrar compras y reposiciones
- ✓ Registrar mermas y ajustes
- ✓ Historial completo por producto

Beneficios Analíticos

1. Reportes Avanzados

- Productos con más salidas/entradas
- Tendencias de consumo por período
- Análisis de rotación de inventario
- Identificación de productos de baja rotación

2. KPIs de Inventario

- Días de inventario disponible
- Tasa de rotación
- Costo promedio de compra
- Margen real vs esperado

3. Optimización de Compras

- Identificar patrones de consumo
- Optimizar frecuencia de reposición
- Reducir sobre-stock
- Evitar quiebres de stock

Beneficios Técnicos

1. Código Limpio

- ☒ Eliminar comentarios "TODO" y código deshabilitado
- ☒ Funcionalidad completa y activa
- ☒ Fácil de mantener

2. Sincronización

- ☒ Schema de Prisma y base de datos alineados
- ☒ Sin inconsistencias

3. Escalabilidad

- ☒ Base sólida para funcionalidades futuras
- ☒ Arquitectura extensible



CONCLUSIONES Y RECOMENDACIONES

Conclusiones

1. Problema Raíz Identificado

- La tabla `inventory_movements` nunca se migró a producción
- El código funcional existe pero está deshabilitado
- Esto causa falta de trazabilidad y auditoría

2. Solución Clara y Directa

- Crear tabla mediante migración de Prisma
- Habilitar código existente (descomentar)
- Mínimos cambios requeridos
- **Alta confianza en la solución**

3. Riesgos Controlados

- Todos los riesgos identificados
- Planes de mitigación y recuperación definidos
- Riesgo general:  ACEPTABLE

4. Impacto en Funcionalidades Existentes

-  **CERO cambios** en funcionalidades actuales
-  **100% retrocompatible**
-  Solo se AGREGAN capacidades nuevas

Recomendaciones

Prioridad 1 (Crítica) - Implementar AHORA

1.  **Aprobar e implementar esta solución**
 - Los beneficios superan ampliamente los riesgos
 - Es la solución correcta arquitectónicamente
 - Resuelve problemas actuales y previene futuros
2.  **Seguir el plan de implementación paso a paso**
 - No saltar fases
 - Validar cada paso antes de continuar
 - Documentar resultados
3.  **Hacer backup antes de iniciar**
 - Obligatorio, no opcional
 - Asegura recuperación rápida si hay problemas

Prioridad 2 (Alta) - Implementar en 1-2 semanas

1.  **Agregar reportes de movimientos en el Dashboard**

```
typescript
// Nueva sección en Dashboard
<Card title="Últimos Movimientos de Inventario">
  <MovementsList limit={10} />
</Card>
```
2.  **Implementar alertas de movimientos anómalos**
 - Ej: Más de X cantidad movida en un día
 - Movimientos sin razón clara
 - Discrepancias entre ventas y movimientos
3.  **Agregar vista de historial por producto**
 - En la página de detalle de producto
 - Mostrar últimos 20 movimientos
 - Filtros por tipo y fecha

Prioridad 3 (Media) - Implementar en 1-2 meses

1.  **Reportes avanzados de inventario**
 - Análisis de rotación
 - Productos de baja/alta rotación
 - Tendencias de consumo
 2.  **Optimización de performance (si necesario)**
 - Batch inserts de movimientos
 - Caching de estadísticas
 - Índices adicionales si se detecta lentitud
 3.  **Integración con otros sistemas**
 - Exportar movimientos a Excel/PDF
 - API para sistemas externos
 - Webhooks de eventos de inventario
-

Prioridad 4 (Baja) - Mejoras futuras

1.  **Consolidar StockAdjustment con InventoryMovement**
 - Actualmente hay dos sistemas paralelos
 - Migrar datos históricos de `stock_adjustments` a `inventory_movements`
 - Deprecar `StockAdjustment` completamente
 2.  **Implementar sistema de “lotes” o “batches”**
 - Para productos con fecha de vencimiento
 - Para trazabilidad nivel lote (ej: farmacéuticas)
 - FIFO/LIFO/FEFO
 3.  **Machine Learning para predicción de stock**
 - Predecir cuándo reponer inventario
 - Optimizar cantidades de compra
 - Detectar anomalías automáticamente
-

Decisión Requerida del Usuario

¿Aprobar implementación de la solución propuesta?

Opciones:

- A.  **APROBAR - Proceder con implementación completa**
 - Seguir plan de implementación
 - Iniciar Fase 1: Preparación
 - Tiempo estimado total: 6-10 horas
- B.  **APROBAR PARCIAL - Implementación por fases**
 - Fase 1: Solo crear tabla (sin habilitar código)
 - Validar por 1 semana
 - Fase 2: Habilitar código después de validación

C. **II POSPONER - Requiere más análisis**

- Especificar qué aspectos necesitan más análisis
- Agendar reunión de discusión

D. **✗ RECHAZAR - Buscar solución alternativa**

- Especificar razones de rechazo
- Proponer alternativa

Métricas de Éxito

Después de la implementación, consideraremos la solución exitosa si:

1. **✓ Funcionalidad básica**
 - Todas las ventas registran movimientos automáticamente
 - API de movimientos funciona (GET y POST)
 - Sin errores en logs de producción
2. **✓ Performance**
 - Tiempo de respuesta de ventas < 1.5 segundos (95 percentil)
 - Sin degradación respecto a versión anterior
 - Cloud Run CPU usage < 70%
3. **✓ Datos correctos**
 - 100% de ventas tienen movimientos correspondientes
 - Stock en `tenant_inventory` coincide con suma de movimientos
 - Sin inconsistencias detectadas
4. **✓ Usabilidad**
 - Usuarios pueden consultar historial fácilmente
 - Usuarios pueden registrar entradas/ajustes sin problemas
 - Feedback positivo de usuarios
5. **✓ Estabilidad**
 - Sin incidentes críticos en primeras 48 horas
 - Sin rollbacks necesarios
 - Monitoreo sin alertas anómalas

SIGUIENTE PASO

Por favor, revisar este documento completo y proporcionar feedback:

1. ¿Hay algún aspecto que necesita más clarificación?
2. ¿Hay preocupaciones adicionales no cubiertas en el análisis de riesgos?
3. ¿Estás de acuerdo con el plan de implementación propuesto?
4. **¿Apruebas proceder con la implementación?**

Una vez aprobado, iniciaré inmediatamente con la Fase 1: Preparación.

Elaborado por: AI Assistant - DeepAgent

Fecha: 21 de Noviembre de 2025

Versión: 1.0

Estado: ⌚ Pendiente de Aprobación del Usuario

Anexos

Anexo A: Archivos a Modificar

```
/home/ubuntu/CRTLPyme/  
└─ prisma/  
    └─ schema.prisma (sin cambios, ya está correcto)  
    └─ migrations/  
        └─ [nueva]_add_inventory_movements_table/  
            └─ migration.sql  
└─ app/  
    └─ api/  
        └─ sales/  
            └─ route.ts (descomentar líneas 271-286)  
        └─ inventory/  
            └─ movements/  
                └─ route.ts (descomentar GET y POST completos)  
└─ PLAN_SOLUCION_INTEGRAL.md (este documento)
```

Anexo B: Queries de Validación

```
-- 1. Verificar tabla existe
SELECT table_name FROM information_schema.tables
WHERE table_name = 'inventory_movements';

-- 2. Contar movimientos por tipo
SELECT type, COUNT(*) as total
FROM inventory_movements
GROUP BY type;

-- 3. Validar que todas las ventas recientes tienen movimientos
SELECT
  s.id,
  s."saleNumber",
  s."createdAt",
  COUNT(im.id) as movimientos_count
FROM sales s
LEFT JOIN sale_items si ON si."saleId" = s.id
LEFT JOIN inventory_movements im ON im.reason LIKE 'Venta ' || s."saleNumber"
WHERE s."createdAt" > NOW() - INTERVAL '24 hours'
GROUP BY s.id
HAVING COUNT(im.id) = 0;
-- Resultado esperado: 0 filas (todas las ventas tienen movimientos)

-- 4. Validar consistencia de stock
SELECT
  ti.id,
  mp.name,
  ti.stock as stock_actual,
  COALESCE(SUM(im.quantity), 0) as suma_movimientos
FROM tenant_inventory ti
JOIN master_products mp ON mp.id = ti."masterProductId"
LEFT JOIN inventory_movements im ON im."tenantInventoryId" = ti.id
GROUP BY ti.id, mp.name, ti.stock
HAVING ti.stock != COALESCE(SUM(im.quantity), 0);
-- Resultado esperado: 0 filas o muy pocas (solo ventas antes de implementación)
```

Anexo C: Comandos de Despliegue

```
# 1. Crear branch
git checkout -b feature/inventory-movements-implementation

# 2. Generar migración
npx prisma migrate dev --name add_inventory_movements_table

# 3. Modificar código (manual)
# Editar app/api/sales/route.ts
# Editar app/api/inventory/movements/route.ts

# 4. Commit changes
git add .
git commit -m "feat: implement inventory movements tracking"

# 5. Push y crear PR
git push origin feature/inventory-movements-implementation

# 6. Merge a main (después de revisión)
git checkout main
git merge feature/inventory-movements-implementation
git push origin main

# 7. Verificar despliegue
gcloud run services describe crtlpyme --region=REGION

# 8. Verificar logs
gcloud logging read "resource.type=cloud_run_revision" \
  --limit=50 \
  --format=json
```

FIN DEL DOCUMENTO